
임의 크기 행렬 곱셈

윤승현

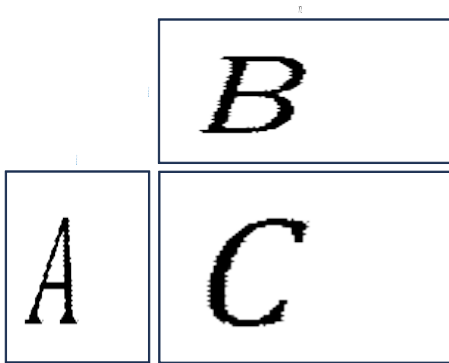
동국대학교 컴퓨터·AI학부

학습 내용

- 임의 크기 행렬의 곱셈을 수행하는 GPU 프로그램
- 행렬 구조체를 사용한 GPU 프로그램

임의 크기 행렬 곱셈

- 입력
 - $A: m \times l, B: l \times n$
- 출력
 - $C = AB: m \times n$



프로그램

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <cuda_runtime.h>
#include <device_launch_parameters.h>

#define TILE_WIDTH 16

__global__ void MatMulGPU(float* C, const float* A, const float* B, int m, int l, int n);
void MatMulCPU(float* C, const float* A, const float* B, int m, int l, int n);

int main()
{
    int m = 1024, l = 512, n = 1024; // A: m x l, B: l x n, C: m x n
    int sizeA = m * l * sizeof(float);
    int sizeB = l * n * sizeof(float);
    int sizeC = m * n * sizeof(float);
    float* A = new float[m * l];
    float* B = new float[l * n];
    float* C_gpu = new float[m * n];
    float* C_cpu = new float[m * n];
    for (int i = 0; i < m * l; ++i) A[i] = (float)(rand() % 3 - 1);
    for (int i = 0; i < l * n; ++i) B[i] = (float)(rand() % 3 - 1);
    for (int i = 0; i < m * n; ++i) C_gpu[i] = C_cpu[i] = 0.0f;
```

프로그램

```
dim3 dimBlock(TILE_WIDTH, TILE_WIDTH);
dim3 dimGrid((n - 1) / TILE_WIDTH + 1, (m - 1) / TILE_WIDTH + 1);

// GPU 실행 시간
clock_t st_gpu = clock();
MatMulGPU << <dimGrid, dimBlock >> > (dev_C, dev_A, dev_B, m, 1, n);
cudaStatus = cudaDeviceSynchronize();
clock_t ed_gpu = clock();
cudaStatus = cudaMemcpy(C_gpu, dev_C, sizeC, cudaMemcpyDeviceToHost);

// CPU 실행 시간
clock_t st_cpu = clock();
MatMulCPU(C_cpu, A, B, m, 1, n);
clock_t ed_cpu = clock();

printf("GPU elapsed time = %.3f ms\n", 1000.0 * (ed_gpu - st_gpu) / CLOCKS_PER_SEC);
printf("CPU elapsed time = %.3f ms\n", 1000.0 * (ed_cpu - st_cpu) / CLOCKS_PER_SEC);

// 결과 비교
double maxError = 0.0;
for (int i = 0; i < m * n; ++i){
    double err = fabs(C_gpu[i] - C_cpu[i]);
    if (err > maxError) maxError = err;
}
printf("Max error = %f\n", maxError);
```

프로그램

```
cudaFree(dev_A);  
cudaFree(dev_B);  
cudaFree(dev_C);  
  
delete[] A;  
delete[] B;  
delete[] C_gpu;  
delete[] C_cpu;  
  
cudaDeviceReset();  
return 0;  
}
```

프로그램

```
__global__ void MatMulGPU(float* C, const float* A, const float* B, int m, int l, int n)
{
    int i = blockIdx.y * blockDim.y + threadIdx.y; // C의 행
    int j = blockIdx.x * blockDim.x + threadIdx.x; // C의 열
    if (i < m && j < n) {
        float sum = 0.0f;
        for (int k = 0; k < l; ++k)
            sum += A[i * l + k] * B[k * n + j];
        C[i * n + j] = sum;
    }
}

void MatMulCPU(float* C, const float* A, const float* B, int m, int l, int n)
{
    for (int i = 0; i < m; ++i) {
        for (int j = 0; j < n; ++j) {
            float sum = 0.0f;
            for (int k = 0; k < l; ++k) {
                sum += A[i * l + k] * B[k * n + j];
            }
            C[i * n + j] = sum;
        }
    }
}
```

테스트 환경

```
Windows PowerShell
Device 0: "NVIDIA GeForce RTX 4070 Laptop GPU"
CUDA Driver Version / Runtime Version      13.1 / 12.1
CUDA Capability Major/Minor version number: 8.9
Total amount of global memory:              8188 MBytes (8585216000 bytes)
(036) Multiprocessors, (128) CUDA Cores/MP: 4608 CUDA Cores
GPU Max Clock rate:                         1605 MHz (1.61 GHz)
Memory Clock rate:                          8001 Mhz
Memory Bus Width:                           128-bit
L2 Cache Size:                              33554432 bytes
Maximum Texture Dimension Size (x,y,z)      1D=(131072), 2D=(131072, 65536), 3D=(16384, 16384, 16384)
Maximum Layered 1D Texture Size, (num) layers 1D=(32768), 2048 layers
Maximum Layered 2D Texture Size, (num) layers 2D=(32768, 32768), 2048 layers
Total amount of constant memory:             65536 bytes
Total amount of shared memory per block:     49152 bytes
Total shared memory per multiprocessor:      102400 bytes
Total number of registers available per block: 65536
Warp size:                                   32
Maximum number of threads per multiprocessor: 1536
Maximum number of threads per block:         1024
Max dimension size of a thread block (x,y,z): (1024, 1024, 64)
Max dimension size of a grid size (x,y,z):   (2147483647, 65535, 65535)
Maximum memory pitch:                        2147483647 bytes
Texture alignment:                           512 bytes
Concurrent copy and kernel execution:        Yes with 1 copy engine(s)
```

테스트 데이터

- 행렬 크기
 - $A: 1024 \times 512$
 - $B: 512 \times 2048$
 - $C: 1024 \times 2048$

TILE_SIZE	4	8	16	32
CPU	2030ms			
GPU	7ms	3ms	3ms	3ms
SpeedUp	290	676	676	676

프로그램(구조체 사용)

```
struct HostMatrix
{
    int rows;
    int cols;
    std::vector<float> elements;

    HostMatrix(int r, int c)
        : rows(r), cols(c), elements(r * c) {}

    float& operator()(int r, int c)
    {
        return elements[r * cols + c];
    }
};
```

```
struct DeviceMatrix
{
    int rows;
    int cols;
    float* elements;
};
```

프로그램(구조체 사용)

```
DeviceMatrix createDeviceMatrix(const HostMatrix& h_M)
{
    DeviceMatrix d_M;
    d_M.rows = h_M.rows;
    d_M.cols = h_M.cols;

    size_t size = h_M.rows * h_M.cols * sizeof(float);
    cudaMalloc((void**)&d_M.elements, size);
    cudaMemcpy(d_M.elements, h_M.elements.data(), size, cudaMemcpyHostToDevice);
    return d_M;
}
```

```
void copyToHost(HostMatrix& h_M, const DeviceMatrix& d_M)
{
    size_t size = h_M.rows * h_M.cols * sizeof(float);
    cudaMemcpy(h_M.elements.data(), d_M.elements, size, cudaMemcpyDeviceToHost);
}
```

```
void freeDeviceMatrix(DeviceMatrix& d_M)
{
    cudaFree(d_M.elements);
    d_M.elements = nullptr;
}
```

프로그램(구조체 사용)

```
int main()
{
    // A: m x l, B: l x n, C: m x n
    int m = 1024;
    int l = 512;
    int n = 2048;
    HostMatrix A(m, l);
    HostMatrix B(l, n);
    HostMatrix C(m, n);

    // 초기화
    for (int i = 0; i < m; ++i)
        for (int j = 0; j < l; ++j)
            A(i, j) = (float)(rand() % 3 - 1);

    for (int i = 0; i < l; ++i)
        for (int j = 0; j < n; ++j)
            B(i, j) = (float)(rand() % 3 - 1);

    // Device 복사
    DeviceMatrix d_A = createDeviceMatrix(A);
    DeviceMatrix d_B = createDeviceMatrix(B);
```

```
DeviceMatrix d_C;
d_C.rows = m;
d_C.cols = n;
cudaMalloc((void**)&d_C.elements, m * n * sizeof(float));
cudaMemset(d_C.elements, 0, m * n * sizeof(float));

// Kernel 실행
dim3 dimBlock(TILE_WIDTH, TILE_WIDTH);
dim3 dimGrid(
    (n - 1) / TILE_WIDTH + 1,
    (m - 1) / TILE_WIDTH + 1);
MatMulGPU << <dimGrid, dimBlock >> > (d_A, d_B, d_C);
cudaDeviceSynchronize();

// 결과 복사
copyToHost(C, d_C);

// 메모리 해제
freeDeviceMatrix(d_A);
freeDeviceMatrix(d_B);
freeDeviceMatrix(d_C);
cudaDeviceReset();
return 0;
}
```

Q&A